

Smart Shortlisting Engine

InternLoom AI Hackathon | Manipal Institute of Technology | 12 September 2026

Purpose

Turn a job description and a batch of resumes into an explainable shortlist. Recruiters can inspect job relevance separately from information that needs human verification. The application runs locally without external AI API keys.

Architecture and matching

React + TypeScript + Vite frontend, **Python + FastAPI** backend. PyMuPDF and python-docx extract text; section parsing preserves evidence. A curated skill dictionary, aliases and TF-IDF provide explicit matching. Local **all-MiniLM-L6-v2** embeddings compare relevant resume sections with job context. A scikit-learn Random Forest provides an additional relevance signal.

Final match score

35% semantic + 30% required-skill coverage + 15% experience/projects
+ 10% preferred-skill coverage + 10% trained-model relevance

Recruiter workflow

Rank and explain: scores for all processed candidates, top-three explanations, matched and missing skills, and candidate comparisons grounded in extracted evidence.

Search and inspect: skill queries such as Python + SQL, deterministic recruiter Q&A, and a requirement matrix with strong, partial and missing states, weighted coverage, filters and evidence popovers.

Review consistency: timeline and experience-claim checks, supporting-certificate comparisons, credential ID extraction and session-scoped review records. Findings do not alter the job-match score.

Handle varied inputs: PDF/DOCX ingestion, section aliases, limited typo tolerance, duplicate-submission grouping and rule-based JD narrow-phrasing flags.

Validation

84 automated tests pass; the full TypeScript/Vite production build passes. The restored live demo returns six candidates and 15 requirements. Earlier stress testing processed 176 PDF/DOCX files from the supplied 220-file dummy dataset; TXT/XML companion files were excluded. These checks validate prototype behavior, not real-world hiring accuracy.

Limitations and safeguards

The relevance model uses 3,000 synthetic examples; real labeled evaluation remains necessary. Rule-based extraction and typo matching can produce errors. OCR is optional and currently unavailable without Tesseract. The prototype has no independent document-authenticity or digital-signature verification, persistent audit storage, or recruiter-account access controls. Requirement confidence values are heuristic, not calibrated probabilities. All flagged information requires human judgment.